🔍 搜索 复习笔记

> 一句话说清搜索：**在一个"问题空间"里一步步试探，最终找到通往目标的路径。** 🗺️ → 🎯

📌 知识地图（先看整体框架）

```
                    搜索问题
                       |
          ┌────────────┴────────────┐
          |                         |
   问题形式化                    搜索策略
   （状态、动作、目标）              |
                        ┌──────────┴──────────┐
                        |                     |
                   无信息搜索              有信息搜索
                   (盲搜)                  (启发式)
                        |                     |
                  ┌─────┴─────┐               |
                  |     |     |               |
                 DFS   BFS   UCS             A* 算法
                                               |
                                          A* 最优性条件
                                          (可采纳/一致性)
```

一、❓ 什么是搜索?

知识点 1：AI 为什么需要搜索?

AI 系统可以分为三大类：

类型	方法	例子
基于算法（ABS） 🧮	数学建模、推公式	线性规划、优化

类型	方法	例子
基于搜索（SBS） 🔍	在解空间中搜索	**AlphaGo/AlphaZero**、路径规划
基于案例（CBS） 📚	找相似历史经验	医疗诊断、推荐系统

> 💡 **SBS 的特点**：不需要理解问题本身，只需要能"试"和"判断好坏"。围棋有 10^{172} 种状态，比宇宙原子数还多 🌌，但 AlphaGo 靠搜索就能找到最优走法。

知识点 2：搜索问题的形式化

一个搜索问题需要定义以下 5 个要素：

要素	符号	含义	例子：在罗马尼亚旅行 🚗
状态空间 🗺️	S	所有可能的状态	各个城市
初始状态 🚩	s_0	开始的位置	Arad
动作集合 🎮	A(s)	在状态 s 可以做哪些操作	开往相邻城市
转移函数 🔄	Result(s,a)	做了 a 后的新状态	Arad→Sibiu
目标测试 🎯	G(s)	判断是否到达目标	到 Bucharest 了吗?
路径代价 💰	c(s,a,s')	走这一步的代价	道路距离（km）

> 搜索的本质：从初始状态出发，通过一系列动作转移到目标状态 🚩 → ... → 🎯

知识点 3：状态空间图 vs 搜索树

```
状态空间图（State Space Graph）🌐          搜索树（Search Tree）🌳

S ──→ a ──→ G                              S
│     │     │                              ┊
│     │     │                           ┌──┴──┐
↓     ↓     ↓                           a     d ...
b ←── c ←── e                             ┊
│     │     │                        ┌──┴──┐
↓     ↓     ↓                        b     a ...
d ──→ p ──→ q
```

	状态空间图	搜索树
节点	所有可能的**状态**	从初始状态出发的**路径**
包含	整个问题空间	探索过的部分
重复节点	每个状态出现一次	可能多次出现（路径不同）

二、🔎 搜索算法的评估维度

知识点 4：评估搜索算法的四个标准

标准	含义	问的问题
完备性 ✅	保证能找到解吗？	只要存在答案，能找到吗？
最优性 🏆	找到的解是最优的吗？	找到的路径是最短的吗？
时间复杂度 ⏱️	运行多快？	要探索多少个节点？
空间复杂度 💾	需要多少内存？	要存多少个节点？

三、🕶️ 无信息搜索（Uninformed Search / Blind Search）

> 核心特点：除了问题定义本身，没有任何额外信息。就像蒙着眼睛找路 😵

知识点 5：深度优先搜索（DFS）⬇️

核心思路：一条路走到黑，走不通再回头。

```
DFS 就像走迷宫 🌀：
选一条路一直走 → 撞墙了 → 退回上一个岔路口 → 换一条路再走
```

实现方式：栈（Stack，FILO）

- OPEN 表（待探索节点）：后进先出

- CLOSED 表（已探索节点）

评估	结果
完备性	❌ 可能陷入无限深分支
最优性	❌ 不一定找到最短路径
时间复杂度	O(b^m)（b=分支因子，m=最大深度）
空间复杂度	O(bm)

知识点 6：广度优先搜索（BFS） ➡️

核心思路：一层一层推进，同一层的节点全部探索完再进入下一层。

```
BFS 就像水波扩散 🌊：
从起点开始 → 先看一步能到哪 → 再看两步能到哪 → 逐层扩大
```

实现方式：队列（Queue，FIFO）

- OPEN 表：先进先出

评估	结果
完备性	✅（如果分支因子有限）
最优性	✅（如果每步代价相同）
时间复杂度	O(b^d)（d=解的深度）
空间复杂度	**O(b^d)**（所有节点都要存 😱）

> 💡 BFS 的最大问题：**空间爆炸**。每层的节点数指数增长，很快就会存不下。

知识点 7：代价一致搜索（UCS） 💰

核心思路：优先扩展**累计代价最小**的节点，而不是深度最浅的。

```
UCS = BFS 的升级版 🚀
BFS 默认每步代价=1，UCS 考虑不同动作的代价不同

类比：去一个地方有好几条路 🗺️
BFS：换乘次数最少的路线
UCS：总路程最短的路线
```

实现方式：优先队列（Priority Queue），按 g(n) 排序

- g(n)：从起点到节点 n 的实际代价
- 每次从 OPEN 表中取出 g(n) 最小的节点扩展

```
UCS 的代价等高线（Cost Contours）：

      ┌── C=1 ── ── C=2 ── ── C=3 ── ...
      │              │              │
      S ─C=3──→ d   e ─C=2──→ ...
      │              │
      └── C=1──→ p ─C=15──→ q ...

UCS 按代价分层扩展，先探索代价小的区域
```

评估	结果
完备性	✅（如果每步代价>0）
最优性	✅
时间复杂度	O(b^(1+⌊C*/ε⌋))
空间复杂度	O(b^(1+⌊C*/ε⌋))

四、💡 有信息搜索（Informed Search / Heuristic Search）

> 核心特点：利用启发式信息（额外的知识）指导搜索方向，避免盲目乱走 🧭

知识点 8：启发式函数（Heuristic Function）

h(n) = 从节点 n 到**目标**的**估计**代价 🎯

> 类比：你在陌生城市找目的地 🗺️
>
> - g(n) = 你已经走了多远（已知）
> - h(n) = 你**估计**还有多远（启发式——可能不准）

评价函数 f(n) = g(n) + h(n)

- g(n) = 从起点到 n 的实际代价（已知道路）
- h(n) = 从 n 到目标的估计代价（启发式——需设计）

知识点 9：最佳优先搜索（Best-First Search）

变体	评价函数	行为
贪心最佳优先 🤑	f(n) = h(n)	只看"离目标多近"，不顾已走距离
UCS 💰	f(n) = g(n)	只看"已走距离"，不顾离目标多远
A* ⭐	f(n) = g(n) + h(n)	**兼顾两者**，最聪明

知识点 10：A* 算法（重点 ⭐）

A* 是搜索算法的"天花板" 🏆

核心思想：把"已经付出的代价"和"未来预计的代价"加起来，选总和最小的节点。

```
f(n) = g(n) + h(n)

g(n) = 从起点到 n 的实际代价      ← 过去
h(n) = 从 n 到目标的估计代价      ← 未来（启发式）
f(n) = 经过 n 的路径的总估计代价
```

A* 的 OPEN 表：按 f(n) 值排序，每次取出 f 最小的节点扩展。

A* 求解八数码问题示例：

```
初始状态 S0              目标状态
 2  8  3                 1  2  3
 1  6  4        →        8  0  4
 7  0  5                 7  6  5

h(n) = 错位数（不在正确位置的数字个数）
g(n) = 已走步数
f(n) = g(n) + h(n)

搜索过程：
S0: g=0, h=5, f=5 → 扩展
├─ A: g=1, h=6, f=7
├─ B: g=1, h=6, f=7
├─ C: g=1, h=4, f=5 → f 最小，先扩展
│   ├─ D: g=2, h=5, f=7
│   ├─ E: g=2, h=4, f=6
│   │   ...
│   └─ K: g=5, h=0, f=5 → 到达目标！🎉
```

知识点 11：A* 的最优性条件（必考 ⭐）

条件一：可采纳性（Admissibility）

h(n) ≤ h*(n) — 启发式函数**不高于**真实代价

> 💡 **h(n) 必须"乐观估计"**，不能高估到目标的距离。
> 比如到目标实际最少还需 10km，h(n) 最大只能是 10，不能是 15。
>
> **结论**：如果 h(n) 是可采纳的，A* 一定能找到最优解 ✅

条件二：一致性 / 单调性（Consistency / Monotonicity）

h(n) ≤ c(n,a,n') + h(n')

> 从 n 到目标的估计代价，不能大于"走一步的实际代价 + 下一步的估计代价"。
>
> **结论**：如果 h(n) 是一致的，A* 在第一次扩展节点 n 时就找到了到 n 的最优路径。

知识点 12：设计更好的启发式

两个启发式函数的比较：

```
h₁(n) = 错误放置的棋子数（简单）
h₂(n) = 每个棋子到目标位置的曼哈顿距离之和（更精确）

对同一个问题：h₂(n) ≥ h₁(n)，因此 h₂ 更好
```

> 💡 **h(n) 越大越好**（但不能超过 h*(n)）。更精确的启发式意味着更少的搜索节点。

Dominance（优势关系）：如果 $h_2(n) \geq h_1(n)$ 对所有 n 成立，那么 h_2 **支配** h_1，用 h_2 搜索的节点数 ≤ 用 h_1 的。

五、🧩 经典搜索问题

知识点 13：TSP（旅行商问题）🌍

问题：给定 n 个城市和城市间的距离，找一条最短路径，**每个城市恰好走一次**再回到起点。

```
例子：5 个城市 A、B、C、D、E

所有可能路径数 = (n-1)! / 2
当 n=5 时：12 种
当 n=30 时：4.9×10²³ 种 → 穷举需要远超宇宙寿命 💀
```

TSP 是 **NP-hard** 问题，大规模时不可能在合理时间内找到最优解。实际中用启发式方法找"足够好"的解。

知识点 14：背包问题 🎒

问题：给定 N 个物品（每个有重量和价格），背包容量有限，怎么装最值钱？

```
物品：  重量(kg)  价值($)
1       4         12        ← 性价比 3
2       2         1         ← 性价比 0.5
3       10        4         ← 性价比 0.4
4       1         1         ← 性价比 1
5       2         2         ← 性价比 1

背包容量：16 kg
搜索空间：2⁵ = 32 种组合
```

知识点 15：Dijkstra 最短路径算法 🛣️

Dijkstra 算法 = A* 的特殊情况（h(n) = 0）

	Dijkstra	**A***
评价函数	f(n) = g(n)	f(n) = g(n) + h(n)
方向性	从起点向所有方向均匀扩展	朝着目标方向扩展
效率	慢（扩展太多节点）	快（有方向感）

Dijkstra 算法步骤：

1. 起点距离 = 0，其他节点 = ∞
2. 取出当前距离最小的节点
3. 更新相邻节点的距离（松弛操作）
4. 重复直到所有节点被处理

📝 自测题

基础概念：

1. 搜索问题的五个要素是什么？用自己的例子解释。
2. DFS 和 BFS 的核心区别是什么？各自用什么数据结构实现？
3. UCS 比 BFS 好在哪？

A* 算法：

4. A* 的评价函数 f(n) = g(n) + h(n) 中，g(n) 和 h(n) 分别代表什么？

5. 什么是 h(n) 的**可采纳性**？为什么它保证了 A* 能找到最优解？
6. 两个启发式函数 h_1 和 h_2，如果 $h_2(n) \geq h_1(n)$ 且都可采纳，哪个更好？为什么？

应用分析：
7. 为什么 TSP 问题不能用穷举法解决？
8. Dijkstra 和 A* 是什么关系？

(可选) 计算题：
9. 八数码问题，计算曼哈顿距离启发式 h(n)：

```
2  8  3
1  6  4
7  0  5     （0 表示空格，目标状态 1-8 顺序排列）
```

📖 推荐阅读

- Russell & Norvig, Artificial Intelligence: A Modern Approach（第 3 章：搜索）
- Dijkstra, A Note on Two Problems in Connexion with Graphs, 1959
- Hart, Nilsson & Raphael, A Formal Basis for the Heuristic Determination of Minimum Cost Paths, 1968（A* 算法）